Gold Layer: The Complete Guide

What the model is, what every cell does, and exactly how the dashboard and orchestration teams should use it

> **Based on:** `nb_p3_batch_job_2.ipynb` (your Gold notebook, 39 cells, with its saved outputs), the Silver document from Member 2, and the Silver check outputs. **What I did and did not do:** I **read** the notebook and its saved results. I did **not** run it. The saved outputs show it ran successfully on your data. **Not in this notebook:** the **alerts fact** you mentioned. Section 9 asks for its columns, and Section 7.6 leaves a slot for it.

Who should read what

You are...	Read
Everyone (5 minutes)	Part 1, Part 2
Understanding the code	Part 3 (cell-by-cell), Part 4 (metric definitions)
Dashboard / Power BI	Part 2, Part 4, Part 5 (column dictionary), **Part 6**
Orchestration / Airflow	Part 2, **Part 7**, Part 8
Before anyone trusts the numbers	**Part 8 (known issues)**

Part 1. The big picture in plain words

1.1 What is "Gold" and why does it exist?

Think of the pipeline as a kitchen:

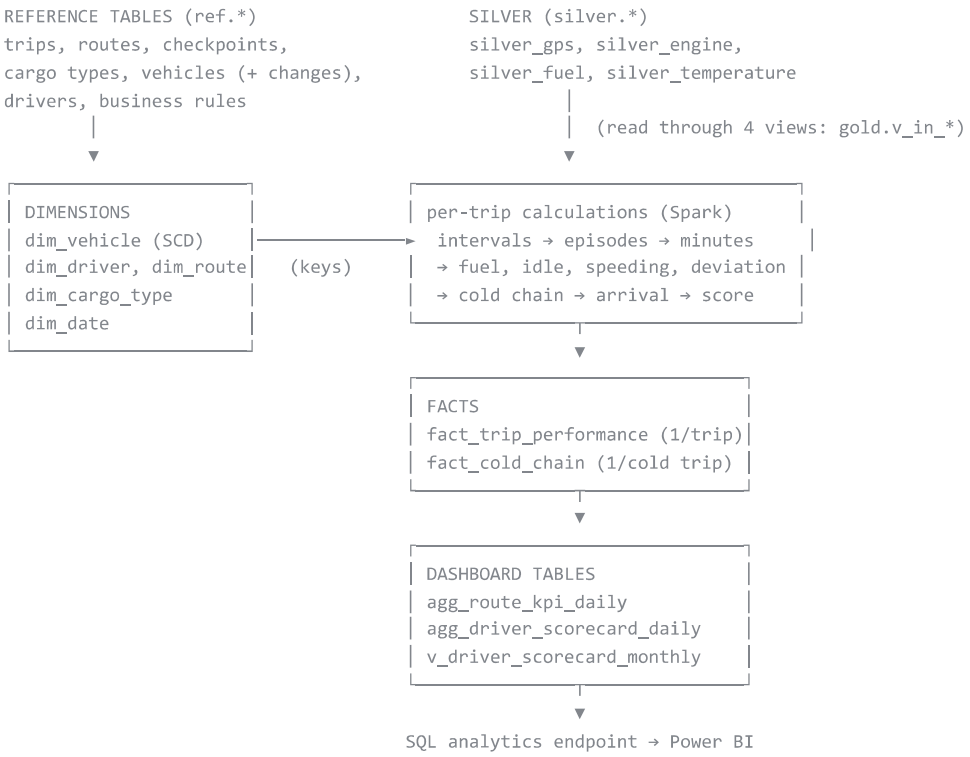
Layer	Kitchen analogy	What it holds
Bronze	Raw groceries	Every message exactly as it arrived (JSON text)
Silver	Washed and chopped vegetables	One clean table per signal (GPS, engine, fuel, temperature)
Gold	**The finished meal on the plate**	**Answers to business questions, one row per trip, ready to put on a dashboard**

Silver still has about **74,000 tiny readings** (one every 10 seconds). Nobody can read that on a dashboard. Gold squeezes them into **one row per trip** (21 rows in your data) with the numbers a manager cares about: fuel used, minutes idling, cargo temperature compliance, late or on time, and a driver score.

1.2 The questions Gold answers

Manager's question	Where the answer lives
Which routes burn the most fuel per 100 km?	`agg_route_kpi_daily` (`fuel_l_per_100km`)
Did refrigerated cargo stay cold? What % of the time?	`fact_cold_chain` , `agg_route_kpi_daily` (`cold_compliance_pct`)
Which cargo was put at risk, and how much money?	`fact_cold_chain` (`cargo_at_risk`)
Which drivers are safe, which idle or speed?	`agg_driver_scorecard_daily` , `v_driver_scorecard_monthly`
Which trips deviated, arrived late, or never arrived?	`fact_trip_performance`

1.3 The data flow



Key idea: Gold never reads Bronze. It reads Silver and the `ref.*` tables only. If Member 2 renames a Silver column, only the four `gold.v_in_*` views change.

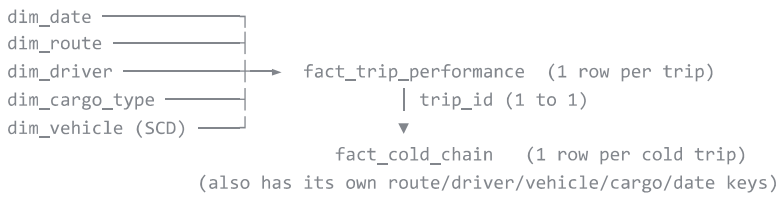
Part 2. What exists in Gold (the map)

2.1 Tables and their grain

Grain = what one row means. Always check this before you use a table.

Table	One row =	Rows now	Type
`gold.dim_vehicle`	one **version** of a vehicle (history kept)	22 (for 20 vehicles)	Dimension, **SCD**
`gold.dim_driver`	one driver	ref_drivers rows	Dimension (overwrite)
`gold.dim_route`	one route	3	Dimension (overwrite)
`gold.dim_cargo_type`	one cargo type	4	Dimension (overwrite)
`gold.dim_date`	one calendar day	about 60 days	Dimension (generated)
`gold.fact_trip_performance`	**one planned trip**	**21**	Fact
`gold.fact_cold_chain`	**one cold-cargo trip** (cold cargo on a refrigerated vehicle)	**14**	Fact
`gold.agg_route_kpi_daily`	one **route x day**	small	Dashboard table
`gold.agg_driver_scorecard_daily`	one **driver x day**	small	Dashboard table
`gold.v_driver_scorecard_monthly`	one **driver x month**	small	**View** (see warning in 8.2)
`gold.v_in_gps / engine / fuel / temperature`	views over Silver	n/a	Internal inputs. **Do not use in dashboards**

2.2 The star schema



Keys you join on (all are big numbers computed from the business ID, called *surrogate keys*):

Fact column	Joins to	Dimension column
`vehicle_key`	`dim_vehicle`	`vehicle_key`
`driver_key`	`dim_driver`	`driver_key`
`route_key`	`dim_route`	`route_key`
`cargo_type_key`	`dim_cargo_type`	`cargo_type_key`
`planned_start_date_key` (e.g. `20260101`)	`dim_date`	`date_key`
`trip_id`	the other fact	`trip_id`

What is a surrogate key? A number made from the real ID with a hash (`xxhash64`). The same input always gives the same number, so rebuilding never changes keys, and no counter table is needed. You never read these numbers; you only **join** on them.

2.3 The SCD (history of vehicles), explained once

The vehicle file arrived twice: an initial file, and a change file at **2026-01-01 12:00**.

Vehicle	What changed	Type	What Gold does
V002	became a **refrigerated truck**	Type 2 (keep history)	**2 rows**: before 12:00 (standard) and after 12:00 (refrigerated)
V010	device changed (`DEV_V010` → `DEV_V010B`)	Type 2	**2 rows**
V007	registration typo fixed (`SIM-O007` → `SIM-0007`)	Type 1 (overwrite)	**1 row**, corrected everywhere

That is why you see **22 versions for 20 vehicles**.

What the dashboard team must know: each trip points to the vehicle version that was **valid at the trip's planned start**. So V002's morning trip (dry goods) is counted under `standard_truck` and its afternoon trip (cold medicine) under `refrigerated_truck`. To count vehicles, count distinct **`vehicle_id`**, never distinct `vehicle_key`.

Part 4. The metrics: definitions you can quote

All times are **UTC**. All numbers are **simulation values**.

4.1 Time-weighted minutes (used by many metrics)

A reading counts for the time until the next reading, capped at 3 x the expected interval. Cap: GPS and engine 30 s, fuel 180 s, temperature 90 s.

4.2 Fuel efficiency

```
fuel_l_per_100km = fuel_used_l / distance_km x 100 .
```

- Per trip: sum of positive changes in the fuel counter and odometer.
- **Per route or day: add up fuel and distance first, then divide** (a ratio of sums). **Never average the trip percentages.**
- Lower is better. Idling fuel is included.

4.3 Cold-chain compliance

```
cold_compliance_pct = in_range_min / (in_range_min + out_range_min) x 100 .
```

- Only **valid measured time** is in the denominator. Sensor faults, spikes, stuck sensors and data holes go to **`unmeasured_min`**, shown separately. They are **not** counted as failures.
- Adherence to the **simulation limits** (e.g. chilled 2 to 8 C), **not** regulatory certification.

4.4 Excursion, breach, compromised

Term	Rule
Excursion	any continuous out-of-range run (any length)
Breach	an excursion lasting **at least** `alert_hold_minutes`
Compromised	the **longest** excursion is **longer than** `max_excursion_minutes` (e.g. chilled 30 min, medicine 15 min, frozen 45 min)
`cargo_value`	cargo quantity x value per unit
`cargo_at_risk`	`cargo_value x spoilage fraction` **only if compromised**, otherwise 0

4.5 Idling

- `idle_min_total` : all time with engine on and speed below 2 km/h.
- `excessive_idle_min` : only idle runs **of 5 minutes or more**.

4.6 Speeding

`speeding_min` : GPS speed above (route limit + 5 km/h). Current limit 90, so above 95.

4.7 Route deviation

`deviation_episodes` : at least 3 consecutive readings (GPS accuracy 50 m or better) more than 500 m from the route line.

4.8 Arrival and lateness

Arrival = first good GPS reading within 1 km of the last checkpoint. `late_min` = minutes after (planned end + grace), floored at 0.

4.9 Trip score and driver score

Each trip starts at **100**. Four penalties are subtracted. For each: *badness* from 0 to 1, times its maximum points.

Penalty	Max points	Badness reaches 1 when...
Speeding	35	speeding minutes are **5%** of driving minutes
Idling	30	excessive idle minutes are **10%** of engine-on minutes
Deviation	20	**2 or more** deviation episodes
Lateness	15	**60 minutes** late (or the trip never arrived = full penalty)

`trip_score = 100 - sum of the four penalties` (one decimal). **Only complete trips are scored.**

Worked example: 12 idle minutes of 400 engine-on gives 9.0 points; 4 speeding minutes of 360 driving gives 7.8; 10 minutes late gives 2.5. Score = 100 - 19.3 = **80.7**.

Driver score (for a day or month) = the average of the driver's trip scores **weighted by driving minutes**. **Bands:** 85 or more **Good**, 70 to 84 **Watch**, below 70 **At risk**.

Part 5. Column dictionary (what each column means)

5.1 `fact_trip_performance` : one row per trip

Group	Columns	Meaning
Keys	`trip_id` ; `vehicle_key` , `driver_key` , `route_key` , `cargo_type_key` , `planned_start_date_key`	identity and joins to dimensions
Business IDs	`vehicle_id` , `driver_id` , `route_id` , `shipment_id` , `cargo_type_id`	readable IDs (use for counting and labels)
Plan	`planned_start_time` , `planned_end_time` , `cargo_quantity` , `trip_status`	from the plan
Actual	`actual_start_time` , `actual_end_time` , `arrived` , `arrival_status` , `late_min`	from GPS (see 4.8)
Fuel	`distance_km` , `fuel_used_l` , `fuel_l_per_100km` , `fuel_counter_anomalies`	NULL when fewer than 2 fuel readings
Engine	`engine_on_min` , `driving_min` , `idle_min_total` , `excessive_idle_min`	minutes
GPS	`speeding_min` , `deviation_episodes`	minutes, count
Score	`score_speeding_penalty` , `score_idling_penalty` , `score_deviation_penalty` , `score_lateness_penalty` , `trip_score` , `score_complete`	NULL unless `score_complete` is true
Quality	`invalid_assignment` , `has_telemetry` , `gps_readings` , `engine_readings` , `fuel_readings`	flags and counts
Lineage	`first_event_time` , `last_event_time` , `built_at`	when data started, ended, and the table was built

Always filter with `has_telemetry = true` for operational KPIs, and `score_complete = true` for scores.

5.2 `fact_cold_chain` : one row per cold trip

`trip_id` , `shipment_id` , the same five keys, `vehicle_id` , `driver_id` , `route_id` , `cargo_type_id` ; **minutes:** `measured_min` , `in_range_min` , `out_range_min` , `unmeasured_min` ; **temperatures:** `min_temperature_c` , `max_temperature_c` (valid readings only); **episodes:** `excursion_episodes` , `breach_episodes` , `longest_excursion_min` ; **flags:** `breach_flag` , `compromised_flag` ; **money:** `cargo_value` , `cargo_at_risk` (currency from `dim_cargo_type` , EGP); **quality:** `suspect_readings` , `temp_readings` ; `built_at` .

5.3 `agg_route_kpi_daily` : route x day (use this for most charts)

`route_id` , `route_name` , `trip_date` ; fuel: `trips` , `trips_with_fuel_data` , `distance_km` , `fuel_used_l` , `fuel_l_per_100km` ; cold: `cold_trips` , `cold_in_range_min` , `cold_out_range_min` , `cold_unmeasured_min` , `cold_compliance_pct` , `breach_episodes` , `excursion_episodes` , `breach_trips` , `compromised_trips` , `cargo_value` , `cargo_at_risk` .

- The cold columns are **blank (NULL) for a route-day with no cold trips**. That is normal.
- `trip_date` = the trip's **planned start date (UTC)**.

5.4 `agg_driver_scorecard_daily` and `v_driver_scorecard_monthly`: driver x day / month

`driver_id`, `driver_name`, `trip_date` (or `month`), `trips`, `driving_min`, `excessive_idle_min`, `speeding_min`, `deviation_episodes`, `late_trips`, `not_arrived_trips`, `score_x_min` (helper: score x driving minutes), `driver_score`, `score_band`.

- Contains **only complete (scored) trips**.

5.5 Dimensions: the labels you put on charts

Dimension	Useful columns
`dim_route`	`route_name`, `origin_name`, `destination_name`, `planned_distance_km`, `speed_limit_kmh`, `allowed_deviation_meters`
`dim_driver`	`driver_name`, `license_category`, `hire_date`
`dim_vehicle`	`registration_number`, `vehicle_type`, `refrigeration_capable`, `device_id`, `effective_from`, `effective_to`, `is_current`
`dim_cargo_type`	`cargo_name`, `temperature_min_c`, `temperature_max_c`, `is_temperature_controlled`, `spoilage_loss_pct` (fraction), `cargo_value_per_unit`, `currency_code`
`dim_date`	`date`, `year`, `month_name`, `day_name`, `is_weekend` (Fri/Sat)

5.6 The alerts fact

Not in the notebook I received. I need its table name and columns before I can document it (Part 9).

Part 6. Dashboard team guide: what to use for what

6.1 Rules before you build anything

1. **Use tables, not the internal views.** `gold.v_in_*` are plumbing; ignore them.
2. **Never average a percentage.** Add numerators and denominators, then divide (examples below). This is the number one mistake.
3. **Blank is not zero.** `fuel_used_l`, `distance_km`, `trip_score`, `cold_compliance_pct` are NULL when unknown. Show them as blank, do not turn them into 0.
4. **Count vehicles by `vehicle_id`**, not by `vehicle_key`.
5. **The dates are January 2026**, not today. Relative date filters ("last 30 days", "this month") will show **nothing**. Use fixed dates or "all".
6. Keep a **footnote** on each page: "*Synthetic simulation data. Times in UTC. Cold-chain compliance means adherence to the simulation thresholds, not regulatory certification.*"
7. Use a **bigger data run** when you design visuals: with 21 trips, charts look empty.

6.2 Power BI model setup

Create the semantic model from the lakehouse SQL endpoint and choose these tables: `fact_trip_performance`, `fact_cold_chain`, `agg_route_kpi_daily`, `agg_driver_scorecard_daily`, `dim_route`, `dim_driver`, `dim_cargo_type`, `dim_vehicle`, `dim_date`.

From (many)	To (one)	Direction
`fact_trip_performance[route_key]`	`dim_route[route_key]`	single
`fact_trip_performance[driver_key]`	`dim_driver[driver_key]`	single
`fact_trip_performance[cargo_type_key]`	`dim_cargo_type[cargo_type_key]`	single
`fact_trip_performance[vehicle_key]`	`dim_vehicle[vehicle_key]`	single
`fact_trip_performance[planned_start_date_key]`	`dim_date[date_key]`	single
`fact_cold_chain[trip_id]`	`fact_trip_performance[trip_id]`	1 to 1 (or use its own keys)
`agg_route_kpi_daily[route_id]`	`dim_route[route_id]`	single
`agg_route_kpi_daily[trip_date]` and `agg_driver_scorecard_daily[trip_date]`	`dim_date[date]`	single

⚠️ Check the exact menu names in your Power BI version. If the SQL endpoint does not show the view `v_driver_scorecard_monthly`, see 8.2.

6.3 Measures (DAX) that are always correct

```
Fuel L per 100 km  = DIVIDE( SUM(agg_route_kpi_daily[fuel_used_l]), SUM(agg_route_kpi_daily[distance_km]) ) * 100

Cold-chain compliance % =
    DIVIDE( SUM(agg_route_kpi_daily[cold_in_range_min]),
            SUM(agg_route_kpi_daily[cold_in_range_min]) + SUM(agg_route_kpi_daily[cold_out_range_min]) ) * 100

Cold-chain coverage % =    -- how much of the time we could actually judge
    DIVIDE( SUM(agg_route_kpi_daily[cold_in_range_min]) + SUM(agg_route_kpi_daily[cold_out_range_min]),
            SUM(agg_route_kpi_daily[cold_in_range_min]) + SUM(agg_route_kpi_daily[cold_out_range_min])
              + SUM(agg_route_kpi_daily[cold_unmeasured_min]) ) * 100

Driver score (weighted) = DIVIDE( SUM(agg_driver_scorecard_daily[score_x_min]), SUM(agg_driver_scorecard_daily[driving_min

Cargo at risk (EGP) = SUM(agg_route_kpi_daily[cargo_at_risk])
Late trips          = SUM(agg_driver_scorecard_daily[late_trips])
```

6.4 Page by page: which table, which fields

Dashboard page	Visual	Table and fields	Notes
Fleet overview	Cards: trips, vehicles, last data time	`fact_trip_performance`: `COUNT(trip_id)`, `DISTINCTCOUNT(vehicle_id)`, `MAX(last_event_time)`	filter `has_telemetry`
	Arrival status split	`fact_trip_performance[arrival_status]`	donut or bar
Route fuel efficiency	Bar: L/100 km by route	`agg_route_kpi_daily` + measure *Fuel L per 100 km* by `route_name`	lower is better
	Table	`route_name`, `distance_km`, `fuel_used_l`, `trips_with_fuel_data`	show blanks
	Trend	measure by `dim_date[date]`	
	Trip drill-through	`fact_trip_performance`: `trip_id`, `fuel_l_per_100km`, `distance_km`, `idle_min_total`	
Cold chain	Card: compliance % and coverage %	measures above	show **both**
	Bar: by route, by cargo	`fact_cold_chain` + `dim_cargo_type[cargo_name]`, or `agg_route_kpi_daily`	
	Table of risky trips	`fact_cold_chain` filtered `breach_flag = true`: `trip_id`, `vehicle_id`, `route_id`, `max_temperature_c`, `longest_excursion_min`, `compromised_flag`, `cargo_at_risk`	
	Temperature range	`min_temperature_c`, `max_temperature_c` against `dim_cargo_type[temperature_min_c/max_c]`	
Driver scorecards	Table with band	`agg_driver_scorecard_daily`: `driver_name`, `driver_score`, `score_band`, `excessive_idle_min`, `speeding_min`, `deviation_episodes`, `late_trips`	conditional colours: Good green, Watch amber, At risk red
	Top and bottom 10	`driver_score` (Top N filter)	
	Drill to trips	`fact_trip_performance` with `trip_score` and the four `score_*_penalty` columns	shows *why* a score is low
Cost impact	Cargo at risk	`fact_cold_chain[cargo_at_risk]`, `cargo_value` by route	currency EGP
Anomaly trends	Idling, deviations, breaches per day	`fact_trip_performance[excessive_idle_min]`, `[deviation_episodes]`; `fact_cold_chain[breach_episodes]` by `dim_date[date]`	counts per trip, **not a per-event list** (see 8.5)
Live operations	Open alerts, vehicle status	the **alerts fact / live status tables**	needs Part 9 info

6.5 Example SQL (SQL endpoint, to check a number before drawing it)

```
-- Fuel efficiency per route (correct: ratio of sums)
SELECT route_name, ROUND(SUM(fuel_used_l)/SUM(distance_km)*100, 2) AS fuel_l_per_100km
FROM gold.agg_route_kpi_daily GROUP BY route_name;

-- Cold-chain compliance per cargo type
SELECT c.cargo_name,
       ROUND(100*SUM(f.in_range_min)/NULLIF(SUM(f.in_range_min)+SUM(f.out_range_min),0), 2) AS compliance_pct,
       SUM(f.unmeasured_min) AS unmeasured_min
FROM gold.fact_cold_chain f JOIN gold.dim_cargo_type c ON f.cargo_type_key = c.cargo_type_key
GROUP BY c.cargo_name;

-- Bottom 10 drivers
SELECT TOP 10 driver_name, driver_score, score_band FROM gold.agg_driver_scorecard_daily ORDER BY driver_score;

-- Trips by vehicle type: shows the SCD effect (V002 appears under both types)
SELECT v.vehicle_type, COUNT(*) AS trips, COUNT(DISTINCT f.vehicle_id) AS vehicles
FROM gold.fact_trip_performance f JOIN gold.dim_vehicle v ON f.vehicle_key = v.vehicle_key
GROUP BY v.vehicle_type;
```

Part 7. Orchestration team guide

7.1 What the Gold job needs and produces

	Details
Needs (inputs)	`silver.silver_gps/engine/fuel/temperature` (after Silver's quality gate passes); all `ref.*` tables; the `gold` schema and tables to exist
Produces	the 5 dimensions, 2 facts, 2 aggregate tables, 1 monthly view
How it loads	**full rebuild** every run: dimensions and facts are **overwritten**; `dim_vehicle` is **MERGEd**. No watermark needed (small data)
Safe to re-run?	**Yes**, except the table-definition cell (see 7.2)
Runs in	a Spark notebook with `lh_fleet` as the default lakehouse

7.2 The notebook is NOT ready to schedule as one block

Problem	Why it matters	Fix (I recommend)
Cell 3 uses `CREATE TABLE` (no `IF NOT EXISTS`) for the two fact tables	A second run **fails**	Move all table definitions into a **one-time setup notebook** (`IF NOT EXISTS`)
The notebook mixes **build** cells and **informational test** cells (many SQL tests just print numbers)	An orchestrator cannot tell pass from fail	Convert checks to `assert` in a separate **validation notebook**
Lots of notebook cells in the middle (demo and diagnostic cells, Gate A)	slow, confusing	Keep only the build cells in the scheduled notebook
No `mssparkutils.session.stop()` at the end	holds trial capacity (earlier HTTP 430 problems)	add it as the last line of each notebook

Recommended split (three notebooks plus the existing Silver job):

Notebook	Contains	Runs
`nb_gold_setup`	schema + `gold.v_in_*` views + table definitions	**once** (and when the contract changes)
`nb_p3_batch_job_2` (build)	dimensions, `dim_vehicle` MERGE, rules, functions, unit tests, facts, aggregates	**every pipeline run**
`nb_gold_checks`	fact checks, referential checks, reconciliation as **asserts**	**every pipeline run, after the build**

I can produce this split version for you as the next step.

7.3 The run order (dependencies)

```
Silver job (Member 2)  ──►  Silver quality gate 1 (must PASS)
        │
        ▼
[1] gold setup (once)  - - ►
[2] Gold build:  dims → dim_vehicle MERGE → rules → unit tests → facts → aggregates
        │            (any assert failing STOPS here)
        ▼
[3] Gold checks (asserts)  ── FAIL → stop, notify, do NOT refresh the dashboard
        │ PASS
        ▼
[4] Refresh SQL endpoint / semantic model  ──►  Power BI shows new data
```

Inside the build notebook the order matters and is already correct: `dim_vehicle` must exist **before** the facts, and facts **before** the aggregates.

7.4 Airflow / pipeline task list

Task	Item to run	Must succeed before
`silver_batch_job_1`	Member 2's notebook	`silver_gate`
`silver_gate`	quality gate 1	`gold_build`
`gold_build`	`nb_p3_batch_job_2`	`gold_checks`
`gold_checks`	`nb_gold_checks`	`refresh_serving`
`refresh_serving`	SQL endpoint metadata / semantic model refresh	`reconcile` , `notify`
`reconcile` , `perf_log`	counts: Bronze to Silver to Gold; durations to `ops_pipeline_run`	end

Use **retries with backoff** (2 retries, growing delay) on every task, as in the project plan. If the Airflow Job cannot be created, build the same chain as a **Fabric Data Pipeline** (the plan's agreed fallback).

⚠️ **Sync delay:** after a notebook writes a Delta table, the **SQL analytics endpoint may need a short time** to show it. Add a small wait or retry before the dashboard refresh and before any T-SQL check.

7.5 Capacity rules (trial)

One Spark notebook at a time; stop the session at the end of every notebook; do not schedule hourly during development (run on demand or a few times a day); keep `smoke` size data while testing.

7.6 What the orchestrator should log after each run

`ops_pipeline_run` row per task (start, end, status), plus these counts: rows in `fact_trip_performance` , `fact_cold_chain` , `agg_*` , number of `arrival_status` values, `fuel_counter_anomalies` , `suspect_readings` , and the **max** `last_event_time` (see 8.1).

Part 8. Known issues and things to verify (honest list)

8.1 ⚠️ Possible future-dated events in Silver (check this first)

What I saw: your Silver check shows event times up to **2026-01-31 / 2026-02-01**, but every trip ends on **2026-01-02**. A gap of about **30 days** (`max_gap_s` = 2,589,240 s) appears in GPS, engine and temperature. These look like the generator's "bad timestamp (future date)" faults. Silver should have rejected them with the "future timestamp" rule, but Member 2's note says that rule is **not applied yet** (it needs the simulated clock).

Why it matters for Gold: such an event sits at the end of a trip's timeline, 30 days after the last real reading. The "hole" logic then adds roughly **43,000 minutes** to `unmeasured_min` for that trip and moves `last_event_time` to late January. The notebook's reconciliation check cannot catch this, because it uses the same (polluted) time span.

Check (run in a notebook):

```
-- trips that have events more than 6 hours after the planned end
SELECT s.trip_id, MAX(s.event_time) AS last_t, t.planned_end_time, COUNT(*) AS late_rows
FROM silver.silver_temperature s JOIN ref.ref_trips t ON s.trip_id = t.trip_id
WHERE s.event_time > t.planned_end_time + INTERVAL 6 HOURS
GROUP BY s.trip_id, t.planned_end_time;

SELECT trip_id, measured_min, unmeasured_min FROM gold.fact_cold_chain ORDER BY unmeasured_min DESC;
```

If it returns rows: (a) ask Member 2 to apply the future-timestamp rule, **or** (b) add a time window to the `gold.v_in_*` views (join `ref_trips` ; keep only `event_time` between `planned_start_time - 30 min` and `planned_end_time + 6 hours`) and rerun. Until fixed, **do not show `unmeasured_min` or coverage on the dashboard.**

8.2 ⚠️ A Spark view may not appear in the SQL endpoint / Power BI

`gold.v_driver_scorecard_monthly` (and the `v_in_*` views) are **Spark views**. I believe views created in Spark are **not visible** to the lakehouse SQL analytics endpoint. **Check:** open the lakehouse SQL endpoint and look for them. If missing: aggregate the **daily table** in Power BI by month (using the weighted measure in 6.3), or recreate the view in T-SQL on the endpoint.

8.4 Small things

Item	Detail	Action
Gate A9 (cell 37)	looks for a rule named `score_limit_lateness_minutes` , but the real name is `score_limit_late_minutes` (the build cell uses the right one)	rename in the query
`arrival_radius_m`	not in `ref_business_rules` ; defaults to 1,000 m	add it to the generator's rules later, or document
All 21 trips `ARRIVED`	the late and never-arrived paths are **not exercised** on this data (late injection is a cut-first feature)	test with a larger or injected run
`invalid_assignment`	cell 33 lists them, expected 0	confirm it prints no rows
Hash keys	`vehicle_key = -1` means "no vehicle version found"	the check asserts none exist
Time zone	UTC everywhere; Egypt is UTC+2	state it on the dashboard

8.5 What Gold cannot show (limits to tell the dashboard team)

- **No list of individual episodes** (start time, end time, peak) in this model. Facts hold **counts and minutes per trip**. A page like "list of breaches with times" needs an episode table (the earlier design had `fact_anomaly_episode`). Your alerts fact may cover it: see Part 9.
- **No per-reading fact.** You cannot plot a trip's temperature curve from Gold. Use Silver for a drill-down if needed.
- **No idle fuel cost** (the plan's cost layer is cut-first). `cargo_at_risk` is the only money figure.

Part 10. Cheat sheet for y'all (one page)

I want to...	Use	Remember
Fuel per route	`agg_route_kpi_daily` , measure *Fuel L per 100 km*	ratio of sums
Compliance %	`agg_route_kpi_daily` , measure *Cold-chain compliance %*	show **coverage** next to it
Which trips breached	`fact_cold_chain` where `breach_flag = true`	`compromised_flag` = real spoilage risk
Money at risk	`cargo_at_risk`	only when compromised
Driver ranking	`agg_driver_scorecard_daily[driver_score]` , `score_band`	only scored trips
Why a driver is low	`fact_trip_performance[score_*_penalty]`	four penalties explain it
Late trips	`late_trips` , `arrival_status` , `late_min`	grace period already applied
Count vehicles	`DISTINCTCOUNT(vehicle_id)`	never `vehicle_key`
Vehicle type on a trip	`dim_vehicle[vehicle_type]` through `vehicle_key`	the type **at trip start**
Date filter	fixed dates 2026-01-01 and later	relative filters show nothing
Run the pipeline	Silver gate → Gold build → Gold checks → refresh	one Spark notebook at a time; stop sessions